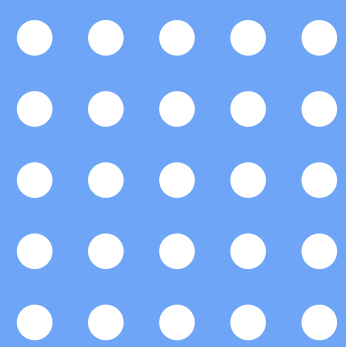
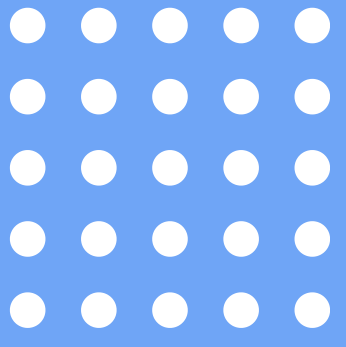




GESTOR DE CLANES Y RAIDS PARA MMORPG

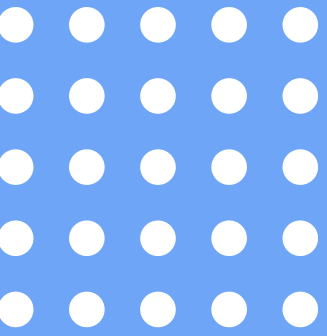
GRUPO 3



FECHA: 04/05/2026



INTEGRANTES



01. SEBASTIÁN SALLES

02. MARTÍN FUENTES

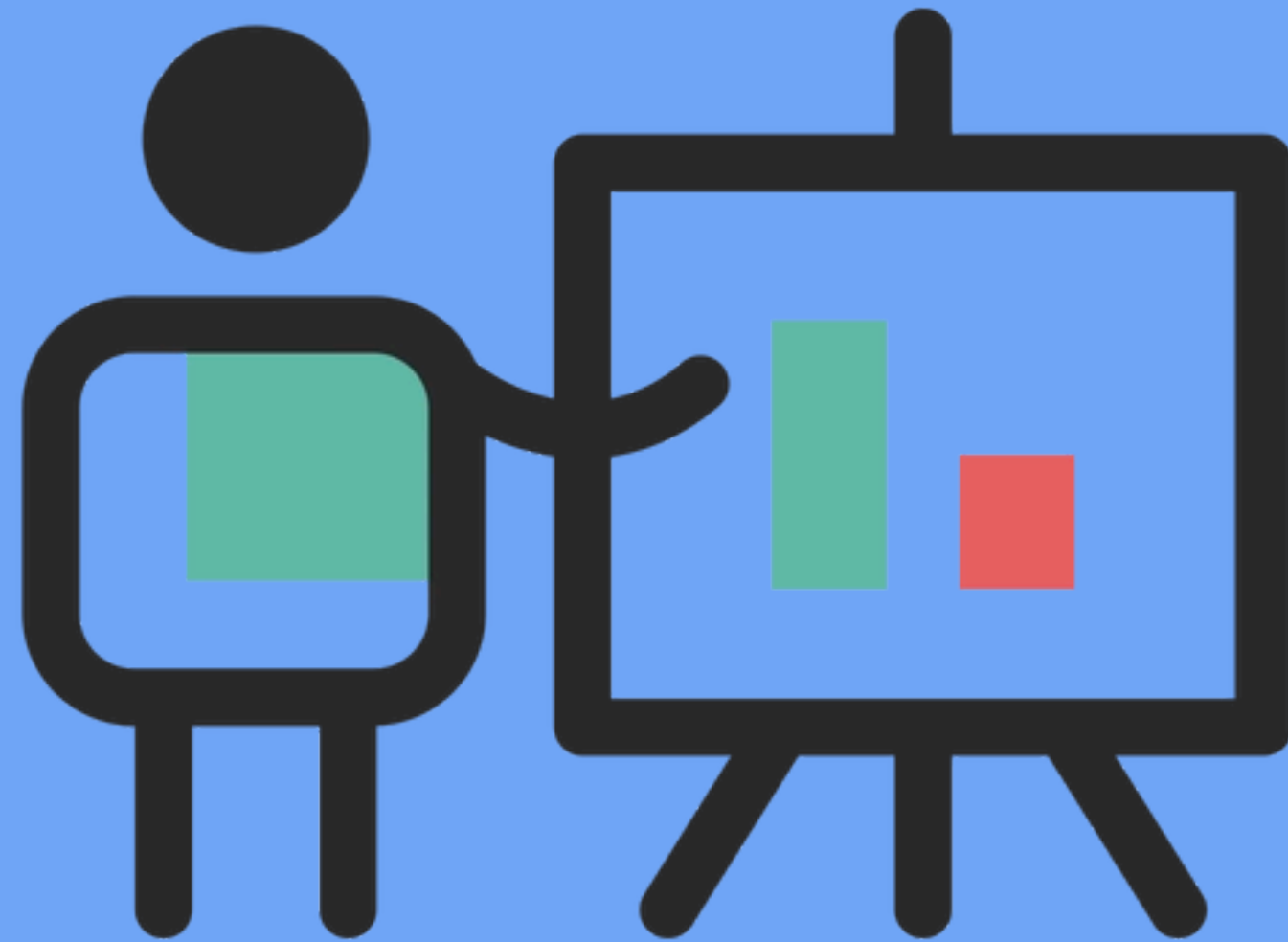
03. IGNACIO CARO

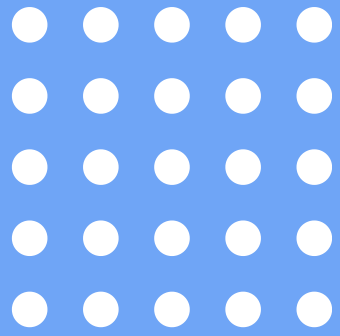
04. DIEGO VEGA

05. ZHI LU

ÍNDICE

- Introducción
- Avance proyecto
- Requisitos
- Conclusión



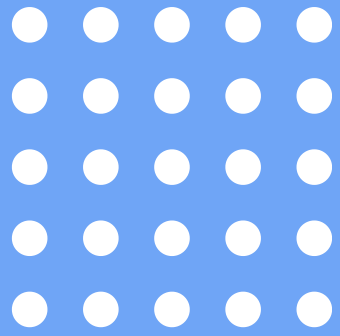


PROBLEMA MMORPG

Una aplicación para gestionar jugadores, sus personajes, eventos (Raids), asistencia y distribución de botín (Loot).

¿QUÉ SE PIDE?

1. API: CRUD de Personajes (Clase, Nivel, Facción) e Items de Botín.
2. API/JWT: El "Guild Master" asigna roles y crea eventos; los miembros normales confirman asistencia.
3. Procedimiento Almacenado 1: Distribuir un ítem de botín a un personaje, deduciendo automáticamente sus puntos de mérito (DKP) del clan.
4. Procedimiento Almacenado 2: Generar un evento de Raid e invitar masivamente a todos los personajes del clan que tengan el rol de "Raider".
5. Trigger 1: Rechazar la inscripción de un personaje a una Raid si su nivel de equipamiento (Item Level) es menor al requerido por el evento.
6. Trigger 2: Auditar la transferencia de liderazgo del clan, guardando quién transfirió el mando a quién y cuándo.
7. Vista Materializada: Un ranking del clan ordenado por asistencia perfecta y contribución, refrescado tras cada Raid.
8. Índices: Indexar la clase del personaje y el ID del evento de Raid para búsquedas rápidas en la formación de grupos.
9. API: Endpoint para ver el calendario de Raids de la semana y los cupos disponibles (Tanques, Healers, DPS).
10. API: Endpoint que muestre el historial de botín de un jugador específico.



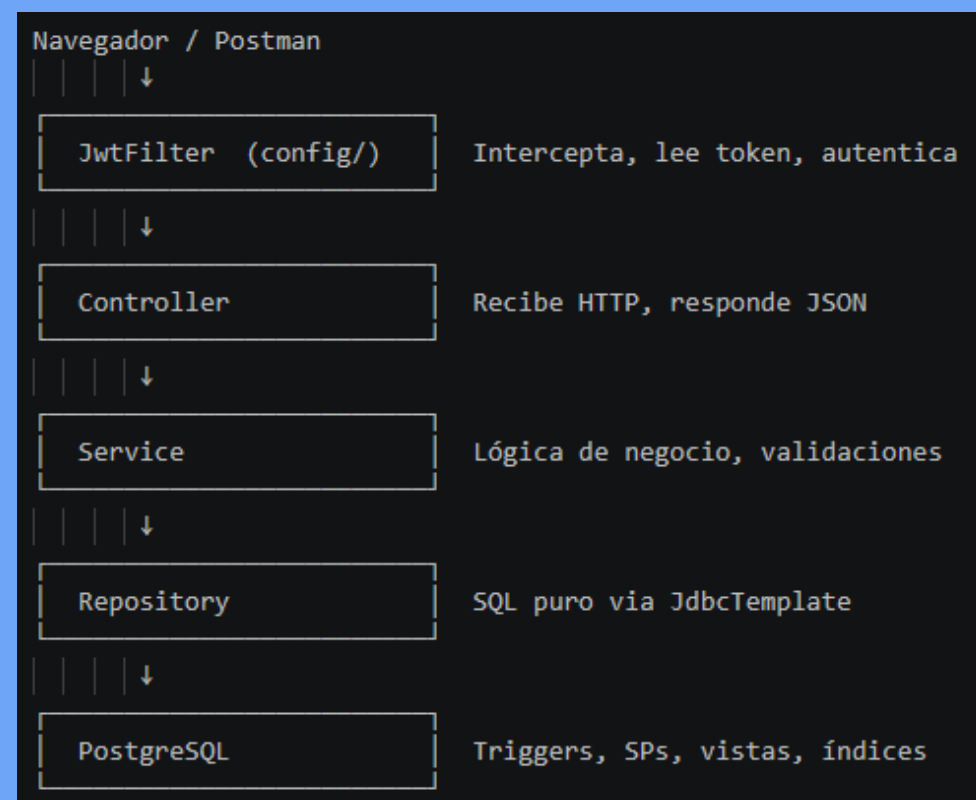
DESCRIPCIÓN **AVANCE DE** **PROYECTO**

Todos los requerimientos fueron implementados correctamente sobre el sistema, haciendo uso de lógicas de negocio como Stored Procedures y Triggers, optimizado a través de vistas materializadas y el uso de index.

Además, gracias al uso de una capa API sólida se asegura la integridad de la información y la seguridad de los datos con el uso de tokens JWT.



1. Cuando se realiza una operación CRUD, ya sea sobre Personajes o Items, el sistema sigue estrictamente una arquitectura por capas donde la información fluye en el siguiente orden:



3. Se implementa paginación en la operación Read (GET), para evitar colapsar la base de datos:

1. CRUD DE PERSONAJES E ITEMS

2. Los métodos HTTP se mapean directamente con acciones SQL para las operaciones CRUD:

1. **Create:** POST -> INSERT.
2. **Read:** GET -> SELECT.
3. **Update:** PUT -> UPDATE.
4. **Delete:** DELETE -> DELETE.

```
// Paginación: LIMIT controla cuantos resultados, OFFSET salta los anteriores.  
// Ejemplo: page=0, size=10 -> LIMIT 10 OFFSET 0 (primeros 10)  
//           page=1, size=10 -> LIMIT 10 OFFSET 10 (siguientes 10)  
public List<Personaje> findAll(int page, int size) {  
    String sql = "SELECT * FROM personaje ORDER BY id_personaje LIMIT ? OFFSET ?";  
    return jdbcTemplate.query(sql, rowMapper, size, page * size);  
}
```



- Representan la barrera de seguridad del sistema, se verifica quién está accediendo y qué es lo que tiene permitido hacer.



- Hay rutas públicas a las que cualquier usuario puede acceder (sin token): "/api/auth/login" y "/api/auth/register".



- Al ingresar las credenciales correctamente, se genera el Token JWT y se lo devuelve al usuario, que funcionará como su credencial para sus futuras peticiones.



- Cada petición es interceptada, se decodifica y verifica su token mediante el filtro JWT y, si es válido, se extrae la información y se la entrega a Spring Security.

- Hay acciones que solo el Guild Master puede realizar y el sistema verifica si la petición corresponde al cargo del personaje. El sistema, además, valida que modificaciones se apliquen solo a personajes pertenecientes al usuario actual.

2. JWT Y ROLES (GUILD MASTER VS MIEMBROS)

```
// Configuración principal de seguridad:
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .cors(cors -> {})
        // Desactivar CSRF porque usamos JWT (no cookies)
        .csrf(csrf -> csrf.disable())
        // No usar sesiones: cada petición se autentica con su token
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // Reglas de acceso:
        .authorizeHttpRequests(auth -> auth
            // /api/auth/** es publico (login y registro)
            .requestMatchers("/api/auth/**").permitAll()
            // Todo lo demás requiere un token válido
            .anyRequest().authenticated()
        )
        // Agregar nuestro JwtFilter ANTES del filtro default de Spring.
        // Así cuando llega una petición, primero pasa por JwtFilter
        // (que lee el token) y después Spring Security decide si dejar pasar.
        .addFilterBefore(new JwtFilter(secretKey),
            UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

```
public void asignarNuevoRol(String usernameAuth, Integer idEjecutor,
    Integer idObjetivo, String nuevoRol) {

    // Validación JWT: ¿el ejecutor es del usuario autenticado?
    if (!personajeRepository.perteneceAUsuario(idEjecutor, usernameAuth)) {
        throw new RuntimeException("No tienes permiso sobre el personaje ejecutor.");
    }

    Personaje ejecutor = personajeRepository.findById(idEjecutor);
    Personaje objetivo = personajeRepository.findById(idObjetivo);

    // ¿El ejecutor es realmente el líder?
    if (!personajeRepository.esLiderDeClan(idEjecutor, ejecutor.getIdClan())) {
        throw new RuntimeException("Solo el Guild Master puede cambiar rangos.");
    }
}
```




- Esta lógica de negocio se ejecuta directamente dentro del motor de PostgreSQL usando un Stored Procedure (SP).



- El repositorio llama a este SP a través del comando CALL y PostgreSQL ejecuta la siguiente secuencia de pasos:

1. Verificar que el ítem existe y obtiene su costo.
2. Verifica que el personaje existe y obtiene sus DKP actuales.
3. Se compara el costo del ítem con el DKP del personaje.
4. Actualiza el DKP del personaje, restandole el costo del ítem.
5. Registrar la entrega en el historial.



- Se ejecuta como una transacción atómica, es decir, si falla un paso se revierten todos los cambios y la cancela.

3.SP1 — DISTRIBUIR BOTÍN

```
-- =====
-- REQUERIMIENTO 3: SP1 - Distribuir botin (deducir DKP)
-- =====
-- ¿Que hace? En una sola transaccion atomica:
-- 1. Verifica que el item existe
-- 2. Verifica que el personaje tiene suficientes DKP
-- 3. Descuenta los DKP del personaje
-- 4. Registra la entrega en historial_botin
-- Si algo falla, todo se revierte (atomicidad).

CREATE OR REPLACE PROCEDURE sp_repartir_botin(
    p_id_personaje INTEGER,
    p_id_item INTEGER,
    p_id_raid INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_costo INTEGER;
    v_dkp_actual INTEGER;
BEGIN
    SELECT costo_dkp INTO v_costo FROM item WHERE id_item = p_id_item;
    IF v_costo IS NULL THEN
        RAISE EXCEPTION 'El item con ID % no existe', p_id_item;
    END IF;

    SELECT puntos_dkp_actuales INTO v_dkp_actual FROM personaje WHERE id_personaje = p_id_personaje;
    IF v_dkp_actual IS NULL THEN
        RAISE EXCEPTION 'El personaje con ID % no existe', p_id_personaje;
    END IF;

    IF v_dkp_actual < v_costo THEN
        RAISE EXCEPTION 'DKP insuficientes (Tiene: %, Necesita: %)', v_dkp_actual, v_costo;
    END IF;

    UPDATE personaje
    SET puntos_dkp_actuales = puntos_dkp_actuales - v_costo
    WHERE id_personaje = p_id_personaje;

    INSERT INTO historial_botin (id_personaje, id_item, id_raid)
    VALUES (p_id_personaje, p_id_item, p_id_raid);
END;
$$;
```




- El repositorio encargado de las raids se comunica con la BD para ejecutar la lógica de negocio que se encarga de crear e invitar a un evento (Raid) a todos los personajes del clan que cumplan los requisitos.



- El flujo lógico interno que se ejecuta en la BD luego de ser invocado desde el backend mediante CALL es el siguiente:

1. Se crea el evento (Raid).
2. Selecciona a todos los personajes del clan que tengan el rol Raider.
3. Se inserta masivamente a todos estos jugadores a la tabla de inscripción de la Raid.



- Durante el paso 3 también se ejecuta el Trigger 1 para verificar el nivel de ítem del personaje a insertar, saltandolo si no cumple con el mínimo establecido.



4.SP2 — INVITACIÓN MASIVA DE RAIDERS

```
-- =====
-- REQUERIMIENTO 4: SP2 - Invitacion masiva de raiders
-- =====
-- ¿Que hace? Recibe un id_raid y un id_clan, y automaticamente
-- inscribe a TODOS los personajes del clan que tengan rol 'Raider'.
-- El Trigger 1 se activa en cada INSERT, asi que si un Raider
-- no tiene suficiente item_level, ESE personaje falla pero los
-- demas se inscriben igual.

CREATE OR REPLACE PROCEDURE sp_invitacion_masiva_raiders(
    p_id_raid INTEGER,
    p_id_clan INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
INSERT INTO inscripciones_raid (id_raid, id_personaje, rol_en_raid, confirmado)
SELECT
    p_id_raid,
    id_personaje,
    'DPS',
    FALSE
FROM personaje
WHERE id_clan = p_id_clan
    AND rol_clan = 'Raider';
END;
$$;
```



- Esta lógica implementa una regla de negocio estricta directamente en la base de datos, garantizando que no se puedan insertar datos inválidos desde ninguna capa de la aplicación.



- El mecanismo se compone de una Función (que contiene la lógica) y un Disparador (que decide cuándo ejecutar esa función) que se ejecuta antes (BEFORE) de insertar.



- Flujo lógico interno del Trigger:
 1. Obtener el ítem level mínimo de la Raid.
 2. Obtener el ítem level del personaje a insertar.
 3. Compara estos valores y, si el ítem level del personaje es inferior, se cancela la inserción.



- Este trigger en conjunto con el SP anterior causa fallos parciales, comportamiento que permite fallar al insertar a un personaje y saltar al siguiente sin cancelar el SP.

5. TRIGGER 1 — VALIDAR ITEM LEVEL

```
-- =====
-- REQUERIMIENTO 5: Trigger 1 - Validar item level en inscripcion
-- =====
-- ¿Que hace? Cuando un personaje intenta inscribirse a una raid
-- (INSERT en inscripciones_raid), este trigger verifica que el
-- item_level del personaje sea >= al item_level_minimo de la raid.
-- Si no cumple, lanza una EXCEPTION y el INSERT se cancela.

CREATE OR REPLACE FUNCTION funcion_validar_item_level()
RETURNS TRIGGER AS $$
DECLARE
    lvl_requerido INTEGER;
    lvl_personaje INTEGER;
BEGIN
    SELECT item_level_minimo INTO lvl_requerido FROM raids WHERE id_raid = NEW.id_raid;
    SELECT item_level INTO lvl_personaje FROM personaje WHERE id_personaje = NEW.id_personaje;

    IF lvl_personaje < lvl_requerido THEN
        RAISE EXCEPTION 'Tu nivel de equipo (%) es insuficiente para esta Raid (Minimo: %)',
            lvl_personaje, lvl_requerido;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_item_level
    BEFORE INSERT ON inscripciones_raid
    FOR EACH ROW
    EXECUTE FUNCTION funcion_validar_item_level();
```



- Este mecanismo implementa un registro histórico de seguridad (Auditoría) a nivel de base de datos, asegurando la trazabilidad de los cambios críticos.



- La secuencia de ejecución ocurre DESPUÉS (AFTER) de que un registro en la tabla de clanes ha sido modificado:

1. Intercepta cualquier intento de UPDATE sobre la tabla clanes.
2. Compara el estado anterior del registro (OLD) con el nuevo estado (NEW).
3. Verifica específicamente si la columna que identifica al líder (id_lider) sufrió una modificación.
4. Si hubo un cambio de líder, inserta un nuevo registro de forma automática en la tabla de auditoría, guardando el ID del líder antiguo, el ID del nuevo líder y el ID del clan.



6. TRIGGER 2 — AUDITORÍA DE LIDERAZGO

```
-- =====
-- REQUERIMIENTO 6: Trigger 2 - Auditar transferencia de liderazgo
-- =====
-- ¿Que hace? Cada vez que alguien cambia el id_lider de un clan
-- (un UPDATE en la columna id_lider de la tabla clanes),
-- este trigger se dispara AUTOMATICAMENTE y guarda en auditoria_liderazgo
-- quien era el lider anterior y quien es el nuevo.
-- OLD = la fila ANTES del UPDATE, NEW = la fila DESPUES del UPDATE.

CREATE OR REPLACE FUNCTION funcion_auditar_liderazgo()
RETURNS TRIGGER AS $$
BEGIN
INSERT INTO auditoria_liderazgo (id_clan, id_lider_anterior, id_lider_nuevo)
VALUES (OLD.id_clan, OLD.id_lider, NEW.id_lider);
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_liderazgo
AFTER UPDATE OF id_lider ON clanes
FOR EACH ROW
EXECUTE FUNCTION funcion_auditar_liderazgo();
```



- Esta estructura almacena físicamente el resultado de una consulta compleja para optimizar drásticamente el tiempo de respuesta al cargar el "Leaderboard".



- El reporte consolida a los jugadores del clan y genera un ranking ordenado según su asistencia y su contribución.



- La secuencia de cálculo y actualización funciona así:
 1. Cruza la información de las tablas de Personaje, Raid e Inscripcion.
 2. Calcula la cantidad de asistencias confirmadas por cada Personaje.
 3. Ordena los resultados priorizando primero la asistencia y luego los puntos de mérito (DKP).
 4. El repositorio de la API (o un trigger) ejecuta el comando REFRESH MATERIALIZED VIEW para actualizar los datos estáticos inmediatamente después de que finaliza cada Raid.



7.VISTA MATERIALIZADA — RANKING DEL CLAN

```
-- =====
-- REQUERIMIENTO 7: Vista Materializada - Ranking del clan
-- =====
-- ¿Que es una vista materializada? Es como una tabla que se genera
-- a partir de un SELECT complejo. A diferencia de una vista normal,
-- los datos quedan GUARDADOS en disco, así que las lecturas son
-- instantaneas. La desventaja es que hay que refrescarla manualmente.

CREATE MATERIALIZED VIEW vista_ranking_clan AS
SELECT
    p.id_personaje,
    p.nombre_personaje,
    c.nombre_clan,
    COUNT(i.id_inscripcion) FILTER (WHERE i.confirmado = TRUE) AS raids_asistidas,
    p.puntos_dkp_actuales
FROM personaje p
    JOIN clanes c ON p.id_clan = c.id_clan
    LEFT JOIN inscripciones_raid i ON p.id_personaje = i.id_personaje
GROUP BY p.id_personaje, p.nombre_personaje, c.nombre_clan, p.puntos_dkp_actuales
ORDER BY raids_asistidas DESC, p.puntos_dkp_actuales DESC;

-- Funcion para refrescar la vista (el enunciado pide logica de refresco)
CREATE OR REPLACE FUNCTION refrescar_ranking_clan()
RETURNS VOID AS $$
BEGIN
    REFRESH MATERIALIZED VIEW vista_ranking_clan;
END;
$$ LANGUAGE plpgsql;
```


8.ÍNDICES



- Se indexa la clase del personaje y el ID del evento Raid para realizar búsquedas rápidas en la formación de grupos.



- Son sentencias escritas directamente en PostgreSQL (CREATE INDEX), y son utilizadas de forma autónoma por su motor.



- Sin índice, la BD se ve obligada a recorrer secuencialmente toda la tabla, mientras que al usar index se aplica una búsqueda B-Tree sobre la estructura de datos externa que PostgreSQL crea con su declaración.



- En este caso, se indexa la clase de los personajes junto a su ID y el ID de las raids, permitiendo de esta forma que se puedan buscar clases específicas para crear grupo y ver la información sobre Raids pasadas casi instantáneamente.

```
-- =====
-- Indice en clase del personaje (Req 8)
-- =====
-- ¿Por que? Cuando el GM arma grupos para una raid, busca por clase
-- ("necesito 2 Paladines, 3 Magos"). Sin indice, PostgreSQL recorre
-- TODA la tabla personaje. Con el indice B-Tree, va directo.
CREATE INDEX idx_personaje_clase ON personaje(clase);

-- =====
-- Tabla de Auditoria de Liderazgo
-- =====
-- Aqui se registra automaticamente cada cambio de lider de un clan.
-- No la llena el backend, la llena el TRIGGER que crearemos mas abajo.
CREATE TABLE auditoria_liderazgo (
    id_auditoria SERIAL PRIMARY KEY,
    id_clan INTEGER,
    id_lider_anterior INTEGER,
    id_lider_nuevo INTEGER,
    fecha_transferencia TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Tabla de Raids
-- =====
CREATE TABLE raids (
    id_raid SERIAL PRIMARY KEY,
    nombre_raid VARCHAR(100) NOT NULL,
    fecha_raid TIMESTAMP NOT NULL,
    item_level_minimo INTEGER NOT NULL,
    tanques INTEGER NOT NULL,
    healers INTEGER NOT NULL,
    dps INTEGER NOT NULL,
    estado VARCHAR(20) DEFAULT 'PROGRAMADA'
);

-- =====
-- Tabla de Inscripciones a Raid
-- =====
-- Relaciona un personaje con una raid. El campo confirmado
-- indica si el Guild Master ya aprobo la asistencia.
CREATE TABLE inscripciones_raid (
    id_inscripcion SERIAL PRIMARY KEY,
    id_raid INTEGER REFERENCES raids(id_raid),
    id_personaje INTEGER REFERENCES personaje(id_personaje),
    rol_en_raid VARCHAR(20),
    confirmado BOOLEAN DEFAULT FALSE,
    fecha_inscripcion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



- Se implementó un endpoint **GET** en la ruta **/api/raids/calendario** que retorna los eventos programados para la semana en curso.



- El cálculo de "Cupos Disponibles" no es un dato estático; la API lo calcula dinámicamente restando los personajes ya inscritos de la capacidad máxima de la Raid.



- La lógica del servicio agrupa esta disponibilidad estructurándola por los roles fundamentales del MMORPG: Tanque, Healer y DPS.



- Esta separación permite al Frontend dibujar la interfaz del calendario informando a los jugadores qué roles hacen falta para completar el grupo.

9. CALENDARIO DE RAIDS CON CUPOS

```
22 // Requerimiento 9: Calendario de raids con cupos disponibles.
23 // Para cada raid, calcula cuantos cupos quedan por rol:
24 // cupos libres = cupos totales de la raid - inscritos en ese rol.
25 // Math.max(x, 0) evita numeros negativos si se sobreinscribieron.
26 public List<RaidDTO> getCalendarioSemanal() {
27     List<Raid> raids = raidRepository.findAll();
28     return raids.stream().map(r -> {
29         int tanquesLibres = r.getTanques()
30             - inscripcionRepository.contarInscritosPorRol(r.getIdRaid(), "TANQUE");
31         int healersLibres = r.getHealers()
32             - inscripcionRepository.contarInscritosPorRol(r.getIdRaid(), "HEALER");
33         int dpsLibres = r.getDps()
34             - inscripcionRepository.contarInscritosPorRol(r.getIdRaid(), "DPS");
35         return new RaidDTO(r,
36             Math.max(tanquesLibres, 0),
37             Math.max(healersLibres, 0),
38             Math.max(dpsLibres, 0));
39     }).collect(Collectors.toList());
40 }
41 }
// Req 9: Calendario con cupos
@GetMapping("/calendario")
public List<RaidDTO> verCalendario() {
    return raidService.getCalendarioSemanal();
}

// Inscribirse a una raid (Trigger 1 valida item level)
@PostMapping("/inscribirse")
public ResponseEntity<?> inscribirse(@RequestBody IncripcionRaid inscripcion,
    Authentication auth) {

    try {
        inscripcionService.anotarPersonaje(inscripcion, auth.getName());
        return ResponseEntity.ok("Inscripcion enviada.");
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
```



- Se recibe una petición GET desde el frontend pidiendo el historial de botín de un personaje por su ID, el controlador de ítems es el que la recibe específicamente.



- El controlador consulta la tabla historial_botin pero al ser una tabla intermedia esta solo registra IDs y fecha, es necesario hacer JOIN para obtener el resto de información.



- ItemRepository contiene la consulta necesaria para devolver la tabla que permite visualizar este historial.



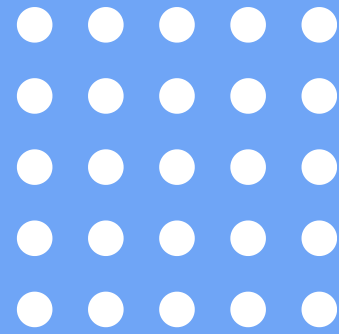
- Finalmente, RawMapper toma esta tabla gigante y empaqueta solo las columnas de interés en un objeto Java que es devuelto al frontend como un JSON limpio.

10. HISTORIAL DE BOTÍN POR JUGADOR

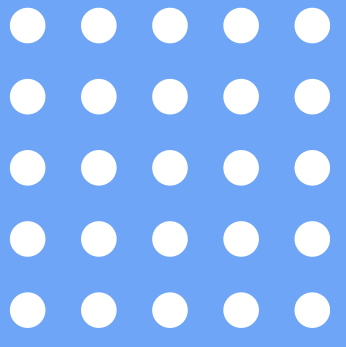
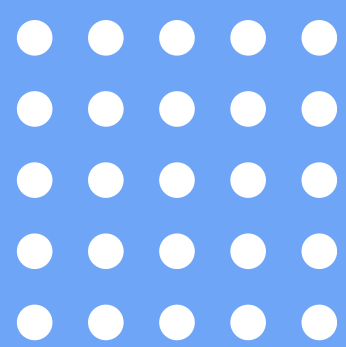
```
-- =====  
-- Tabla de Historial de Botin  
-- =====  
-- Cada vez que se reparte un item, queda registrado aqui.  
-- La llena el Stored Procedure sp_repartir_botin.  
CREATE TABLE historial_botin (  
    id_entrega SERIAL PRIMARY KEY,  
    id_personaje INTEGER REFERENCES personaje(id_personaje),  
    id_item INTEGER REFERENCES item(id_item),  
    id_raid INTEGER REFERENCES raids(id_raid),  
    fecha_entrega TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
// Requerimiento 10: historial de botin de un jugador.  
// Hace JOIN con item y personaje para traer los nombres  
// en vez de solo los IDs (mas legible para el frontend).  
public List<HistorialBotin> obtenerHistorialPorPersonaje(Integer idPersonaje) {  
    String sql = "SELECT h.*, i.nombre_item, p.nombre_personaje " +  
        "FROM historial_botin h " +  
        "JOIN item i ON h.id_item = i.id_item " +  
        "JOIN personaje p ON h.id_personaje = p.id_personaje " +  
        "WHERE h.id_personaje = ?";  
    return jdbcTemplate.query(sql, (rs, rowNum) -> {  
        HistorialBotin h = new HistorialBotin();  
        h.setIdEntrega(rs.getInt("id_entrega"));  
        h.setIdPersonaje(rs.getInt("id_personaje"));  
        h.setIdItem(rs.getInt("id_item"));  
        h.setIdRaid(rs.getInt("id_raid"));  
        h.setNombreItem(rs.getString("nombre_item"));  
        h.setNombrePersonaje(rs.getString("nombre_personaje"));  
        h.setFechaEntrega(rs.getTimestamp("fecha_entrega").toLocalDateTime());  
        return h;  
    }, idPersonaje);  
}
```


CONCLUSIÓN



- Se levanta exitosamente una base de datos conectada a un backend y un frontend que simula el sistema que gestiona jugadores, items, raids y clanes de forma segura mediante el uso de tokens JWT.
- Trabajar en todos los aspectos de una aplicación/sistema, ser capaz de levantarlo y realizar las queries necesarias para satisfacer los requerimientos fue un gran desafío, pero se aprecia que una base sólida ayuda a que el resto del trabajo funcione de manera más expedita.
- Queda pendiente trabajar más en el frontend y realizar preguntas al cliente para dejar claro que es lo que realmente busca para lograr generar la query apropiada.



**GRACIAS POR SU
ATENCIÓN!**